

PLATFORM ADMINISTRATOR GUIDE

HospitalOS

Running the AgoroX platform side of HospitalOS

For AgoroX Technologies staff

Version 1.0 · July 2026

AgoroX Technologies, Ibadan

CONFIDENTIAL — for AgoroX Technologies staff only. Do not share this document with tenant hospitals or external parties.



Contents

- 1** Accessing the Platform view
- 2** Tenant management
- 3** Settlement centre
- 4** Usage analytics & churn signals
- 5** Platform health & feature flags
- 6** Architecture for support staff
- 7** RBAC and the audit chain, explained for support
- 8** Known limitations & the roadmap to production
- 9** Escalation & support



1 Accessing the Platform view

Exactly one account carries platform admin rights in this build:

Field	Value
Email	support@agorox.africa
Password	agorox
Role	Super Admin, with platformAdmin: true

Signed in on that account, a **Hospital / Platform** toggle appears in the topbar. Switching to Platform replaces the hospital sidebar entirely — this is a different context, not a section of the hospital app, and no hospital-side staff account can reach it.

Important. This credential is hardcoded for the preview build. Before any real tenant goes live, this must move to a properly authenticated, MFA-protected account — see Section 8.

2 Tenant management

The Platform Overview lists every tenant deployment with plan, seats, active users, status, and monthly recurring revenue. Current seed tenants, for reference when testing:

Tenant	Plan	Seats	Active	Status	MRR
Ibadan Teaching Hospital	Enterprise	240	186	Active	■850,000
Lagoon Specialist Clinic	Standard	40	31	Active	■180,000
Jos Community Hospital	Trial	15	9	Trial	—
Kano Medical Centre	Standard	60	0	Suspended	—
AgoroX Demo	Lifetime	5	2	Active	—

A tenant can be suspended or reactivated from this screen. Suspending a tenant does not delete its data — it blocks sign-in for that tenant's users until reactivated.

Note. Kano Medical Centre is seeded as Suspended with zero active users on a Standard plan — a deliberate example of exactly the situation Section 4's churn detector is built to catch earlier, before a tenant reaches this state.

3 Settlement centre

AgoroX takes a platform fee of **3.25%** on each tenant's total collections, settled on a monthly cycle.

The cycle lifecycle

Pending → Processing → Settled. A failed cycle can be retried, which returns it to Processing. Every cycle records gross collected, the fee taken, the net paid to the hospital, and a settlement date once paid.

Note. The fee and net amounts are computed directly from the gross figure and always reconcile exactly — fee + net = gross, on every cycle, with no rounding drift. This was verified directly against the settlement service before shipping: overriding a fee rate and re-running a cycle recomputes both figures consistently.



Payout destination

The settlement screen shows the configured payout bank account and processor (Paystack) that receives AgoroX's fee each cycle.

4 Usage analytics & churn signals

Per-tenant metering — seats, active users, encounters, lab orders, storage, API calls — is the evidence behind every invoice and the basis for plan-sizing conversations at renewal.

Seat under-utilisation

A tenant using less than half its purchased seats is flagged explicitly as a churn risk. This is not decorative: a tenant paying for capacity it is not using is a tenant re-evaluating the relationship at renewal, and it is far cheaper to have that conversation early than to lose the account cold.

When you see a flagged tenant, the right response is a proactive conversation about whether their plan still fits their actual usage — not waiting for them to raise it.

5 Platform health & feature flags

The health panel shows the services HospitalOS depends on, each with a status, latency, and uptime figure:

Service	Role
Web app (Cloudflare Pages)	Serves the built frontend
API Worker	Backend request handling
D1 database	Persistent tenant data (not yet wired — see Section 8)
R2 object storage	File storage (not yet wired — see Section 8)
HL7 instrument gateway	Live analyzer/device connections (simulated in-app today)
Email (Resend)	Transactional email delivery

Feature flags let a plan tier unlock or withhold specific modules per tenant — useful for phased rollouts (e.g. enabling Radiotherapy only for tenants with an oncology unit) without a separate deployment.



6 Architecture for support staff

You do not need to read code to support tenants, but the shape below explains why certain things behave the way they do when a tenant reports an issue.

Everything is in-memory right now

Every module — patients, orders, prices, documents, everything — runs on an in-memory data layer. A tenant's data resets when their browser session ends. This is deliberate for this preview stage, and every service is written as an async function shaped exactly like a real network call, specifically so the eventual swap to Cloudflare D1 requires changing the inside of each service function, not the screens that call it.

“Engines” vs. modules

Two things in the codebase are deliberately structured as standalone engines rather than ordinary feature modules — Help and Pricing. An engine owns its own state, is imported *by* other modules, and never imports a module itself. This is why a price override set in Administration → Pricing takes effect immediately across every billing calculation and every screen that displays that price, from one single source of truth, rather than needing to be updated in five different places.

The instruments gateway is a simulation of live listening

Device management, registry, and monitoring are real. What is simulated is the network listener itself — an MLLP socket for HL7, a DICOM SCP for imaging, a print daemon — because a browser cannot open a network listening socket. Every “receive/confirm/send” action in the interface calls the exact same function a live listener would call, so when the real listener is built server-side, nothing downstream (Lab, Radiology, Alerts) needs to change.

7 RBAC and the audit chain, explained for support

Two-level permissions

Every role has an AREA list (which sidebar groups it can open) and a GRANTS list (which specific actions it can perform within those areas, as “area:action” strings, e.g. patient-care:discharge). When a tenant reports “my nurse can’t discharge a patient,” that is very likely by design — check the role matrix under Administration → Users & roles before treating it as a bug.

The audit chain

Every consequential action — sign-ins, permission denials, clinical actions, price changes, claim decisions — is written to an append-only, hash-chained log. Each entry links to the hash of the one before it. If a tenant asks whether their audit log can be trusted, the honest answer is: it is tamper-*evident*, not tamper-proof, in this browser-only build — anyone with developer tools could still alter in-memory state directly. Its real value is architectural: the identical chain design runs unchanged once records move to server-side append-only storage, where it becomes a genuine, enforceable control.

8 Known limitations & the roadmap to production

Read this before promising a prospective tenant anything about production-readiness.



Area	Current state	What production needs
Data persistence	In-memory, resets on reload	Cloudflare D1 per tenant
Authentication	Client-side credential check	Server-verified session (JWT/D1)
File storage	Session-only blob URLs	Cloudflare R2
Audit chain	Tamper-evident in-browser	Server-side append-only store
Lab/imaging devices	Simulated message flow	Real MLLP socket + DICOM SCP (Worker)
Platform admin login	Hardcoded credential	MFA-protected authenticated account

Note. None of this is a defect in what has been built — every service function is already shaped for exactly this migration. It is scoped, known work, not architecture that needs to be redone.

9 Escalation & support

For issues that are genuinely product bugs rather than expected RBAC behaviour or known preview limitations (Section 8), escalate to the AgoroX engineering channel with: the tenant name, the account role in use, the exact screen and action, and whether the issue reproduces on a fresh sign-in.

Note. This guide is distributed only through the Platform view of the application, visible solely to the platform admin account. It is not linked anywhere a tenant user could reach.